Thotsakan Statistics

ทศกัณฐ์ สถิติ

A guide to the documentation

What each document says, how the app is built, and how to work on it

For: Irving Gomez (lab head) and the students who work on the project.

Describes: the code and documents on branch `irving-reviewing-experiment`, 27 October 2026, the day before v1 is deployed (Wednesday 28 October 2026).

How to use it: this guide is a map. The documents in the repository are the source of truth; when they change, they win over this PDF.

Contents: 1 The app in one page · 2 The documents · 3 Decisions and rules · 4 Architecture · 5 Inside a tab · 6 Working on the project · 7 Status and what comes next · 8 Glossary

1 The app in one page

Thotsakan Statistics is a web **statistics analysis tool** for CMKL students and staff, in the style of jamovi or JASP. You load a dataset, pick a variable and the options you want, and the tables, plots and formulas update by themselves. The app reports numbers and draws plots; **the user decides**. It never prints a verdict such as "Reject H_0", and it has no lessons or teaching flow.

The name comes from the giant Thotsakan, who has ten faces and twenty arms to see and control the world from every angle: the app gives its users "ten arms" (sliders, switches, selectors) to change the variables and watch the data respond.

Tab	What it does
Home	Welcome page.
Data	Upload a comma-separated CSV file, or open one of three example datasets (World Happiness Report 2023 — the default, Howell, Victoria electricity demand 2014). Fix column types, filter categories, set display precision.
Probability › Common Distributions	PMF/PDF of 12 distributions, with the area of a query such as $P(X \le 5)$ or $P(2 < X \le 5)$.
Statistics › Graphical Analysis	Histogram or ECDF of a variable, with KDE, a normal model and a confidence band.
Statistics › Descriptive Statistics	Quantiles, centre, spread and shape, with estimator tags (Unbiased / Consistent); histogram and box plot.
Statistics › Statistical Inference	Confidence and prediction intervals (normal theory and bootstrap); confidence regions for (μ, σ).
Hypothesis Testing	One- and two-sample t, Bartlett / Levene, one-way ANOVA; α, critical values, rejection region. Works, but needs a deep rework after v1.
Linear Regression	Planned.

v1 is the version deployed on 28 October 2026 and shown at the event on 6 November. It covers the Data tab and the four tabs Common Distributions, Graphical, Descriptive and Inference, all made consistent with each other. Hypothesis Testing stays as it is; Linear Regression and other file formats come later.

2 The documents: what each file holds

All documentation lives in the folder `docs/`, plus a few files at the top of the repository. Read them in the order of the first column when you join the project.

Read	File	What it holds — and when to open it
1	`README.md`	The front page: what the app does, the feature table, how to install and run it, how to run the tests, and the list of documents. *Open first.*
2	`docs/decisions.md`	Every decision on how the app looks and behaves, dated, taken by Irving Gomez, with the commits that built it and, where recorded, the reason. *Open before changing anything visible.*
3	`docs/interaction_contract.md`	The 17 rules every analysis tab follows (options, automatic results, tables and plots, naming, layout, colors). *Open when you build or change a tab.*
4	`docs/design.md`	What the app is, the three panels, the DualInput control, the theme and the performance budget. Also the long-term idea of a guided-teaching layer (not built).
5	`docs/architecture.md`	How the frontend and backend fit together: layers, folders, request flow, sessions, filters, every API endpoint, testing and CI. *Open when you write code.*
6	`docs/roadmap.md`	The authoritative feature status, the one backlog after v1 (in priority order), the Linear Regression plan, migration notes and the open known issues. *Open when you look for work.*
—	`docs/AUTH.md`	How to set up Google sign-in (restricted to `@cmkl.ac.th`) and fix sign-in problems.
—	`docs/plans/2026-10-23-v1-release.md`	The checklist of the steps to v1 (A done, B done, C documentation, D release, E protection). Removed after the deploy.
—	`CONTRIBUTING.md`	Workflow, coding standards, how to test, how to write commits and pull requests.
—	`AGENTS.md`, `CLAUDE.md`	Instructions for AI coding assistants (Codex, Cursor, … read `AGENTS.md`; Claude Code reads `CLAUDE.md`, which points to it): the same rules, written for them.
—	`CODE_OF_CONDUCT.md`	How we treat each other.
—	Branches	`main`: the released app the server runs. `dev`: the next release. One short branch per task, merged into `dev` by a pull request Irving approves, then deleted. The professor's original Gradio app is archived under the tag `gradio-final`.
—	`changelogs/`	One dated note per batch of changes: what changed, in plain words.

One fact, one place. Each kind of information has exactly one home: decisions in `decisions.md`, rules in the contract, feature status and backlog in `roadmap.md`, how the code works in `architecture.md`. Other documents link there instead of repeating it, so nothing goes out of date twice.

3 Decisions and rules: what nobody changes alone

The rule. Nobody — student or AI agent — reverses or works around a decision recorded in `docs/decisions.md`. If the change you want conflicts with one, stop and talk to Irving first. A decision changes only through a new dated entry in that file, written or approved by Irving. The interaction contract changes the same way: ask Irving, record the decision, change the contract, then the code.

The decisions that shape everything

- **An analysis tool, not a guided lab.** The app reports; the user decides. No verdicts, no lessons.
- **Results update on their own.** About 250 ms after a change (700 ms while bootstrap is on). No Run button, only Reset — and Reset shows the result for the defaults straight away.
- **Three panels, always the same order.** Left: inputs only (what to show first, then one Settings section). Center: tables at the top, plots below. Right: the formulas of what is on screen, then a closing "Values" section.
- **Formulas are copied from the code** in `backend/core/`, never from a textbook, and only for what was chosen.
- **One meaning per color.** In every plot: mean **blue**, median **green**, deviation **amber**, prediction **rose**, KDE **purple**, normal model **orange**. All colors live in `frontend/src/plot/theme.ts`.
- **Readable text.** Small text is 14 px; nothing, plots included, is under 12 px.
- **Data.** CSV only for now. A column is numeric when at least 80% of its filled cells are numbers; the other cells count as missing. Filters compare values as written in the CSV.

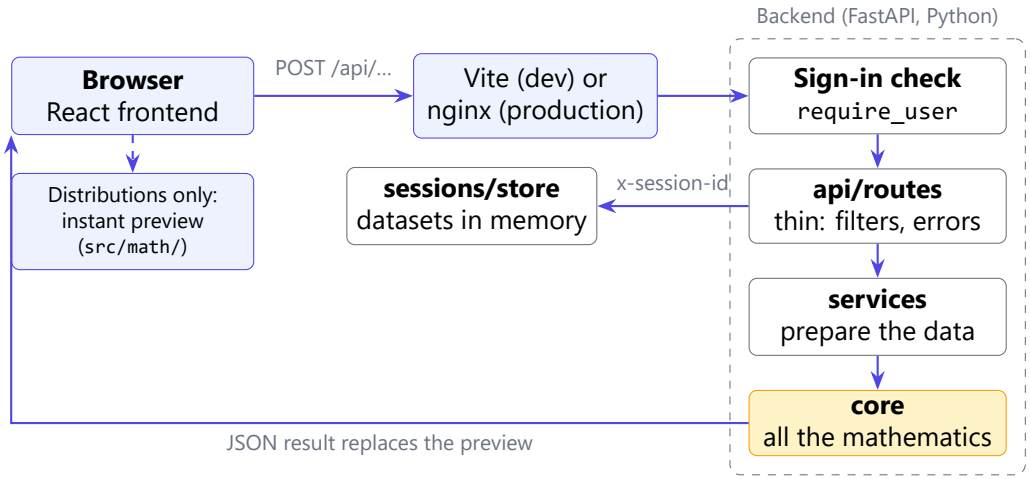
The full list (31 entries) with dates and commits is in `docs/decisions.md`.

The contract in five lines

1. **Options:** a hidden option is never sent; show an option only when it applies; never force an input; sensible defaults ticked; options that only change the display never recalculate.
2. **Running:** automatic, debounced; one shared status component for computing, errors and warnings.
3. **Results:** adding a method adds rows (bootstrap next to normal theory); the program reports, the user decides; warnings only where they apply.
4. **Naming:** one display-name map (`labels.ts`); α is always "Significance level α"; discrete probabilities are bars.
5. **Layout and states:** shared components and shared colors, never per-tab copies.

4 Architecture

The app has two halves. The **frontend** (React, in `frontend/`) draws the screen. The **backend** (FastAPI in Python, in `backend/`) does every statistical calculation. The professor checks the mathematics by reading `backend/core/`.



4.1 What happens when you move a slider

1. You change a setting in the left panel. The tab's state hook waits about 250 ms for you to stop.
2. In Common Distributions only, the browser first draws an instant *preview* with its own copy of the formulas (`frontend/src/math/`). It must give the same numbers as the backend.
3. The hook cancels any older request still on its way and sends a new one (`POST /api/...`) with the dataset's session id in the `x-session-id` header.
4. The backend checks you are signed in, finds your dataset in memory, applies the Data tab's filters, and calls the service, which prepares the data and calls `core/`.
5. `core/` computes at full precision; the result comes back as JSON and is drawn. Rounding happens only on screen.

4.2 The backend in layers

Each layer may only call the layer below it. This keeps the mathematics in one place the professor can check.

Folder	Job and rules
`api/routes/`	One file per area. Thin: apply the filters, call the service, turn a `ValueError` into a 400 error. Every router gets its URL prefix in `main.py`.
`api/schemas/`	The shape of each request and response (Pydantic models).
`api/deps.py`	Shared helpers: the sign-in check, finding the dataset (`get_session_data`), and the one filter every tab uses (`apply_filters`).
`services/`	Prepare the data (missing values, weights) and map results to schemas. May *not* import numpy, scipy or statsmodels.
`core/`	All mathematics, at full precision. Imports nothing from the rest of the project. Files copied from the professor's original app are kept byte-identical; new behaviour goes in new files next to them.
`sessions/store.py`	Uploaded datasets in memory, 30 minutes after last use; a restart clears them. That is why the backend runs a single worker.

4.3 The frontend folders

Folder	What is inside
`features/`	One folder per tab (`data/`, `probability/common/`, `estimation/graphical/`, `estimation/descriptive/`, `estimation/inference/`, `hypothesis/`).
`components/`	Shared pieces every tab uses: `StatusBanner`, `ResultTable`, `ResultSection`, `CheckboxTree`, `MathFormula` (KaTeX), `DualInput`, form controls, export menu.
`plot/theme.ts`	Every plot color and the shared Plotly look.
`labels.ts`	One display name for every statistic and estimator.
`api/`	One small file per backend area, with the request and response types.
`context/`	`DataContext` (the dataset, its session id, column types, filters) and `AuthContext` (sign-in).
`layout/`	The page frame: logo bar, header, sub-header, the three-panel `LabBench`, footer.
`math/`	The distributions preview only. No other statistics in the frontend.

4.4 Datasets, sign-in and filters

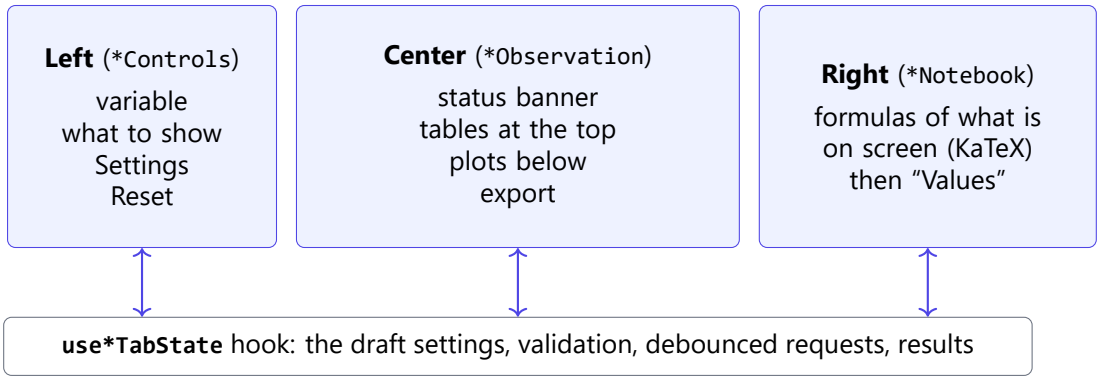
- **Sign-in:** Google, restricted to verified `@cmkl.ac.th` accounts; the backend sets a signed cookie. Setup in `docs/AUTH.md`.
- **Datasets:** the Data tab reads the CSV in the browser (for the preview and the filter lists) and also uploads it. The backend keeps it in memory under a session id, which the frontend sends with every analysis request. Both sides use the same 80% rule to decide which columns are numbers.
- **Filters:** chosen in the Data tab, sent with each request and applied on the server by one shared function, so every tab analyses exactly the same rows.

4.5 The API at a glance

Endpoint	Purpose
`GET /api/health`	Is the backend alive?
`/api/auth/…`	Sign-in configuration, Google sign-in, current user, sign-out.
`POST /api/data/upload`	Upload a CSV; returns the dataset's session id and column types.
`POST /api/probability/compute`	One distribution: PMF/PDF, the query's probability, mean and variance.
`POST /api/descriptive/compute`	Descriptive statistics, histogram and box plot data.
`POST /api/inference/estimators`	Which estimators suit a column.
`POST /api/inference/ci`, `/pi`, `/regions`	Confidence intervals, prediction intervals, confidence regions.
`POST /api/graphical/compute`	Histogram or ECDF with its overlays.
`POST /api/hypothesis/test`	One of four tests with α, critical values and rejection region.

5 Inside a tab

Every analysis tab is built the same way, so once you know one you know them all.



- The three panels are exported from one `*Tab.tsx` file and placed into the shared `LabBench` layout by `App.tsx`. The layout never changes; a tab only fills the three slots.
- The `use*TabState` hook is created in `App.tsx`, so a tab keeps its settings when you switch away.
- Separate *request* settings (they change the calculation) from *display* settings (they only change what is shown and never recalculate).
- Use the shared components and `plot/theme.ts`; do not write a per-tab status box, table style or color.

6 Working on the project

6.1 Run it

Command (from the repository root)	Gives you
`docker compose -f docker-compose.dev.yml up`	Development stack with hot reload: app on `localhost:5173`.
`docker compose up --build`	Production-style stack: app on `localhost:8080`.
`npm install && npm run dev`	Without Docker: frontend on :5173, backend on :8000.

The backend refuses to start until sign-in is configured in the root `.env` file (see `docs/AUTH.md`); on your own machine you may set `AUTH_DISABLED=true` instead.

6.2 Test it — before every commit

`cd backend && pytest`	Backend tests (about 220).
`cd frontend && npm run test`	Frontend tests (about 250).
`cd frontend && npm run build`	Type check and production build.

GitHub runs the same checks on every pull request, but run them yourself first and report the real result.

6.3 Change it

1. Read `decisions.md` and the interaction contract. If your idea conflicts with either, talk to Irving first.
2. Write a test that shows the problem or the new behaviour; see it fail.
3. Make the change in the right layer: mathematics in `core/`, data preparation in `services/`, screens in `features/`, shared look in `components/` and `plot/theme.ts`.
4. Run the three test commands above.
5. Update the documents the change touches: the feature table in `roadmap.md` and `README.md`, a dated note in `changelogs/`, and `decisions.md` if Irving took a new decision.
6. Commit with a Conventional Commit message (`feat:`, `fix:`, `docs:`, …) and open a pull request.

Never: add statistics to the frontend; edit the professor's original files in `core/`; invent reference values for tests (check them against scipy, statsmodels or a cited source); commit a real dataset or a secret; add a dependency without approval.

7 Status and what comes next

7.1 Steps to v1

Step	State on 27 October 2026
A. Consistency additions	Done (intervals table, formulas, colors, plotting rules).
B. Correctness and review	Done (weights bug, final review of the branch).
C. Documentation	C1 `docs/` rename, C2 decision log, C3 documents updated: done. C4 one version number (1.0.0, semantic versioning): done.
D. Release	Verification and merge into `dev`: done. Deploy on Wednesday 28 October.
E. Protecting the decisions	`AGENTS.md` and tests in CI: done. Branch protection on GitHub: next.

7.2 After v1

The backlog in `docs/roadmap.md`; the first three in the order Irving set:

1. A goodness-of-fit test for the normal distribution, shown at the top of the left panel wherever a result assumes normal data.
2. A deep rework of Hypothesis Testing.
3. Linear Regression.
4. Classifying variables (continuous, discrete, ordinal, nominal, …) instead of guessing from the values.
5. Data tab: Excel and SPSS files; better handling of very large datasets.
6. Backend tidy-up, small interface items, and later the custom distributions, approximations, Thai / English text and the guided-teaching layer.

8 Glossary

Backend / frontend	The Python server that computes / the React web page that shows.
Session id	The key under which the backend keeps your uploaded dataset in memory.
Debounce	Waiting a short moment after the last change before sending a request, so a burst of changes becomes one request.
Provisional preview	The instant curve drawn by the browser in Common Distributions, replaced by the backend's result.
Request vs display setting	One that changes the calculation vs one that only changes what is shown.
Interaction contract	The 17 rules every analysis tab follows (`docs/interaction_contract.md`).
Decision log	The dated list of Irving's decisions (`docs/decisions.md`).
Conventional Commits	Commit messages that start with a type: `feat:`, `fix:`, `docs:`, `refactor:`.
CI	Continuous integration: the GitHub workflow that builds the app when `dev` changes.